Automated Emergency Alert System

A practical implementation roadmap for a mobile SOS / crash-alert app with live maps, nearby emergency-facility discovery, and authorized responder notification.

What it does	What it must not assume
Detects or receives an SOS/crash signal, shares a location, finds nearby facilities, and routes alerts through approved channels.	A map search does not dispatch an ambulance. A hospital listing does not authorize direct alerts. Public emergency numbers are not general-purpose APIs.

Safety and deployment boundary

Build and test first as a non-emergency prototype with simulated incidents and consenting test users. Before real-world use, obtain written approval and technical specifications from the relevant emergency communications authority, police/EMS dispatch, hospitals, and privacy/regulatory stakeholders. Never send test alerts to live responders.

This guide is a software engineering plan, not medical, legal, or dispatch advice. Emergency numbers, dispatch workflows, data-protection rules, and authorized integrations vary by location.

Recommended first release

- A user-initiated SOS app with optional, clearly consented crash detection.
- A map showing the user's position and nearby verified emergency facilities.
- Alerts to opted-in household/workplace contacts; dispatch integration remains disabled until formally approved.
- A responder dashboard that records acknowledgement, escalation, and incident closure.

1. System architecture

Keep detection, incident management, facility discovery, and dispatch as separate components. This makes the app useful in a prototype while preventing an unapproved map or messaging integration from being mistaken for public-safety dispatch.

Layer	Suggested technology	Responsibility
Mobile app	Flutter or React Native; native location/sensor modules where required	SOS, consent, countdown/cancel, location capture, map, status and fallback instructions.
API / incident service	Python FastAPI or TypeScript NestJS	Authenticate device, validate incident, deduplicate events, create state, apply escalation policy.
Data and jobs	PostgreSQL + PostGIS; Redis-backed queue	Persist incidents, facility records, consent and audit; deliver retries and scheduled escalation.
Map and facility search	Google Maps Platform, HERE, or an OSM-based provider	Display map, geocode, search nearby facilities, and optionally calculate routes.
Notifications	Push provider plus an approved SMS/voice provider	Notify opted-in contacts and staff; keep delivery status and retry policy.
Responder console	Web app with role-based access	Acknowledge, view incident context, assign responder, resolve, and audit changes.
Operations	Cloud-managed database/queue, secrets manager, monitoring	Encryption, availability, incident response, retention and operational visibility.

A future live-dispatch adapter should be a separate, feature-flagged integration. Its provider, message format, certification, authorization, coverage, and fallback are jurisdiction-specific; confirm them directly with the relevant authority.

Event-to-response flow

1. Trigger	2. Verify	3. Enrich	4. Notify	5. Close
SOS tap or sensor event	Countdown; cancel; validate	GPS + incident + verified facilities	Approved contacts / authorized dispatch adapters	Responder acknowledgement; tracking stops on authorized closure

Facility discovery is supporting context only. Dispatchers and qualified professionals determine the appropriate destination and response; do not auto-route a patient based only on distance, scraped web data, or a guessed injury classification.

2. End-to-end workflow

A. Trigger and false-alarm handling

- For a manual SOS, require an intentional action and show the current location and the recipients who will be notified.
- For crash detection, start with an on-device candidate-event detector and a visible/audio countdown. Let the user cancel or confirm; permit a configured no-response escalation only after local safety review.
- Use a unique incident ID, event timestamp, detection source, and idempotency key. Repeated sensor messages must not create duplicate incidents or repeated dispatch.
- If the phone is offline, present clear local emergency instructions, attempt the permitted device-level communication path, and queue non-dispatch data. Do not claim help was contacted unless delivery is confirmed.

B. Location and nearby facilities

- Request location only with explicit permission; capture coordinates, accuracy radius, timestamp, and whether the fix is stale. Ask for a landmark/address when GPS is poor.
- Search a bounded radius for verified emergency departments / trauma facilities using a licensed or compliant map/place API. Show distance, source, and last-verified time; give the user a route link.

- Use provider routing/traffic data only where licensed. A route estimate is not a guarantee of an open emergency department, bed availability, or clinical suitability.
- Keep an authoritative facility directory and contact metadata from a health authority, hospital network, or contracted data source. Do not scrape hospital websites to discover private ER lines or operational status.

C. Alert and escalation

- Notify the user's opted-in emergency contacts and authorized on-site team; ask recipients to acknowledge receipt.
- Escalate through an official, contracted, jurisdiction-approved dispatch interface only when available. Do not make automated calls/texts to public emergency numbers or police from a consumer app.
- Send a hospital pre-arrival message only through an approved hospital/network endpoint and only under a documented data-sharing agreement. Otherwise provide facility details to the authorized dispatcher/responders.
- When a motor-vehicle crash is confirmed, police notification follows the local dispatch authority's process. Do not notify police solely because an unverified sensor threshold fired.
- Continue location updates only while the incident is active and the required permission/legal basis remains valid. Stop tracking on authorized resolution or user cancellation where policy permits.

3. Step-by-step build plan

Step	Build	Done when
0. Define scope	Choose one launch jurisdiction and user group. Document emergency authority, dispatch contacts, privacy rules, supported phones, and who can resolve incidents.	Named owners approve workflows, recipients, consent, retention, and a live-test prohibition.
1. Prototype the UX	Build SOS, permission onboarding, cancellation countdown, status, contact setup, and map screen. Use mock data and a simulation mode.	A usability test shows users understand when an alert is sent and whether it was acknowledged.
2. Add location/maps	Integrate map SDK and place search. Add location freshness/accuracy, manual address entry, provider attribution, and no-result/offline states.	Tested coordinates and facility results are accurate for sample locations; no scraping is used.
3. Build backend	Implement sign-in/device registration, incident creation, state transitions, PostGIS facility queries, audit trail, and idempotency.	Duplicate/replayed requests are safe; access is tenant-scoped and audited.
4. Add contact alerts	Send push notifications to opted-in contacts; optionally add an approved messaging provider with rate limits, delivery callbacks, and retries.	Delivery/failed states are visible; provider outage and duplicate delivery are tested.
5. Add crash detection	Collect consented sensor features, develop a threshold/classifier using controlled data, and tune for device differences. Keep manual SOS independent.	False-positive/false-negative rates meet a pre-agreed target in a monitored pilot; users can cancel.
6. Add responder console	Implement role-based queue, acknowledgement, assignment, map/context view, escalation timer, closure reason, and audit history.	Only authorized personnel can access or change incidents; actions are traceable.
7. Integrate public safety	If approved, implement the authority's specified CAD/NG911/other adapter in a test environment, including authentication, certification, acknowledgement, retries and fallback.	Authority signs off on conformance and an end-to-end exercise; feature flag remains off otherwise.
8. Pilot and operate	Run tabletop drills, field tests with consenting participants, monitoring, on-call procedures, training, and post-incident review.	Documented go-live approval, rollback plan, support owner and periodic re-validation.

Suggested MVP sequence: ship steps 0–4 in simulation/contact-only mode; treat sensor detection and any direct dispatch as later, separately gated releases.

4. Data model and API sketch

Keep sensitive medical profile fields out of the first prototype unless they are essential, legally permitted, consented, and protected. Minimize each alert to data an authorized recipient needs.

Record	Example fields
Incident	id, user_id, source (sos/crash), status, created_at, location_lat, location_lon, accuracy_m, location_at, crash_confidence, idempotency_key
Contact	user_id, recipient_id, verified_channel, consent_at, priority, enabled
Facility	id, name, category, coordinates, authoritative_source, source_updated_at, public_contact, emergency_capability (verified/unknown)
Delivery	incident_id, recipient_type, channel, provider_message_id, state, attempted_at, acknowledged_at, failure_code
Audit event	incident_id, actor_id, action, timestamp, reason, request_id

Example incident payload (use synthetic values in development)

```
{ "incident_id": "generated-uuid", "event": "crash_suspected", "occurred_at": "2026-01-15T10:20:30Z",
"location": { "latitude": 0.0000, "longitude": 0.0000, "accuracy_m": 18, "captured_at": "2026-01-15T10:20:25Z"
```

```
}, "user_confirmation": "no_response", "callback_reference": "protected-reference", "medical_profile_ref": null
}
```

Example API surface: POST /v1/incidents (create/idempotent); GET /v1/incidents/{id} (authorized status); POST /v1/incidents/{id}/acknowledgements; POST /v1/incidents/{id}/resolve; GET /v1/facilities/nearby?lat=...&lon;=...&radius;_m=.... Validate inputs, authenticate every call, enforce ownership/roles, rate-limit, and avoid exposing precise location in public URLs or logs.

5. Safety, privacy, and reliability

Concern	Design requirement
False alarms	Visible cancel/confirm interaction; configurable countdown; deduplication; clear cancellation propagation; monitor false-positive rate.
Missed or late alert	Show delivery state and timestamp; bounded retry/backoff; alternate approved contact path; explicit offline behavior; no success-shaped fallback.
Bad/stale location	Attach accuracy and capture time; update while active with consent; allow address/landmark correction; flag stale or low-confidence GPS.
Dispatch authorization	Use only formally authorized interfaces and approved test endpoints. Never use scraped phone numbers or automate calls to public emergency lines.
Privacy	Data minimization; explicit consent; purpose limitation; defined retention/deletion; access/export procedure; assess applicable privacy laws before launch.
Security	TLS in transit; managed encryption at rest; short-lived tokens; least privilege; MFA for console; secrets manager; dependency patching; audit logs.
Abuse and spoofing	Rate limits, device/account abuse detection, signed webhooks, replay protection, recipient verification, and an incident-reporting route.
Map provider failure	Display last known location and timestamp; provide manual address/call instructions; don't present unavailable facility data as current.
Service outage	Redundant monitoring, queue health checks, tested backup/restore, documented operator escalation, and an honest app status.

Do not collect blood type, allergies, medications, or detailed medical history by default. If a future feature is approved, use a minimum necessary, user-reviewed profile; tightly control recipient access, encrypt it, and define retention and correction processes.

6. Testing and launch checklist

- Unit: incident state machine, countdown/cancel, idempotency, access control, radius search, stale-location rules, notification policy.
- Integration: map/provider timeouts, contact delivery callbacks, retries, duplicate webhooks, queue restart, database failover, unauthorized actors.
- Device: permissions denied, background restrictions, low battery, weak GPS, no network, OS reboot, supported models and battery impact.
- Scenario: false sensor trigger; confirmed crash; no-response countdown; wrong location; no facility results; responder acknowledgement; cancellation race; resolution.
- Security/privacy: threat model, authorization tests, log review for sensitive data, retention/deletion test, consent withdrawal, backup access control.
- Exercise using test recipients and simulator mode. Verify the app never contacts live emergency services during development or staging.
- Before launch: written authorization, approved jurisdiction and service area, responsible operations team, training, support plan, go/no-go checklist, rollback, and post-incident review.

7. First build milestone

A safe first milestone can be completed without live dispatch: create an app with manual SOS, a 10–15 second cancel/confirm countdown, live location and accuracy, map-based nearby verified facilities, and alerts to consenting test contacts. Mark every screen and message as a simulation until an authorized responder integration is formally approved.

Definition of done for the prototype

- A test incident can be created once, viewed on a map, and closed by an authorized test responder.
- The user sees who receives the alert and whether it was delivered or acknowledged.
- Facility results show a source and verification date; no website scraping is part of the pipeline.
- No path sends a call, SMS, or alert to police, EMS, or a hospital's operational staff.
- Synthetic user and location data are used in development, and logs contain no unnecessary medical information.

Important: In a real emergency, use the local emergency number or existing official emergency channel. This software guide does not replace emergency services.